

# CSE 225L Lab 05

## Unsorted List

### Array-Based

Implementation, Dry Runs, Debugging, and Complete Solutions in C++

A premium lab guide for building the List ADT with a fixed-size dynamic array, then testing it with integers and student records.

COURSE

**CSE 225L Data Structures  
and Algorithms Lab**

LANGUAGE

**C++**

CORE ADT

**UnsortedType<T>**

BUILD TARGET

**main.cpp + given class files**

# 1. Lab Map

This lab builds an **unsorted array-based List ADT**. The list stores items in the order they are inserted, but it does not keep them sorted. The engine underneath is a fixed-size heap array created by `new T[SIZE]`, controlled by three private members: `data`, `currentSize`, and `pointTo`.

Template class

Array-based ADT

Linear search

Swap-with-last deletion

Student objects

| What this lab covers                                                  | Why it matters                                                                                  |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Creating and testing <code>UnsortedType&lt;T&gt;</code>               | You learn how a reusable ADT hides array details behind clean operations.                       |
| Insertion, searching, deletion, traversal, reset                      | These are the bones of almost every list implementation you will meet later.                    |
| Using a template class with <code>int</code> and <code>Student</code> | The same data structure can store multiple data types if the type supports required operations. |
| Operator overloading for <code>Student</code>                         | Delete and Search need <code>==</code> , so the object must know what equality means.           |

# 2. Page / Slide Extraction Map

| Page | Extracted Content                                                                        | Important Details                                                                                                                                                                    |
|------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Lab title and objectives                                                                 | Design List ADT, implement array-based unsorted list, create <code>UnsortedType</code> , test insertion/search/deletion/reset, create <code>Student</code> .                         |
| 2    | <code>unsortedtype.h</code> and source beginning                                         | Class fields: <code>T *data</code> , <code>currentSize</code> , <code>pointTo</code> . Constructor allocates <code>new T[SIZE]</code> . Destructor uses <code>delete[] data</code> . |
| 3    | <code>IsFull</code> , <code>MakeEmpty</code> , <code>Insert</code> , <code>Search</code> | Insertion only checks full condition. Search uses linear scan and a reference boolean.                                                                                               |
| 4    | <code>Delete</code> , <code>GetNext</code> , <code>Reset</code>                          | Deletion replaces deleted item with the last item, so order can change. Traversal uses internal cursor <code>pointTo</code> .                                                        |
| 5    | Complete lab tasks and expected output                                                   | Integer list tasks must match given sequence. Student class must store ID, name, CGPA and print records after deleting ID 15467.                                                     |

### Extraction status

All five PDF pages were readable. No task, code fragment, or table was marked unreadable.

## 3. Implementation Toolkit

### 3.1 What This Lab Is Really About

This lab is not just about inserting numbers into an array. It is about building a tiny data-structure machine: private memory inside a class, public functions as buttons, and a driver program that presses those buttons in the exact order required by the lab sheet.

The list is called **unsorted** because deletion and insertion do not preserve any mathematical order. The code only cares about storage and retrieval, not arranging values.

### 3.2 Concepts Before Code

| Concept          | Lab-specific meaning                                                                    | Watch this                                                                                   |
|------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Template class   | <code>UnsortedType&lt;T&gt;</code> can store <code>int</code> or <code>Student</code> . | The type <code>T</code> must support default construction, assignment, and <code>==</code> . |
| Dynamic array    | <code>data = new T[SIZE]</code> creates 5 slots on the heap.                            | Destructor must use <code>delete[] data</code> , not <code>delete data</code> .              |
| Logical size     | <code>currentSize</code> counts valid items.                                            | The array capacity is 5, but valid values are from index 0 to <code>currentSize - 1</code> . |
| Internal cursor  | <code>pointTo</code> remembers where traversal is.                                      | Call <code>Reset()</code> before printing the list.                                          |
| Student equality | Two students are treated equal if their IDs match.                                      | Deleting ID 15467 works by creating a temporary <code>Student</code> with that ID.           |

### 3.3 How to Recognize the Coding Pattern

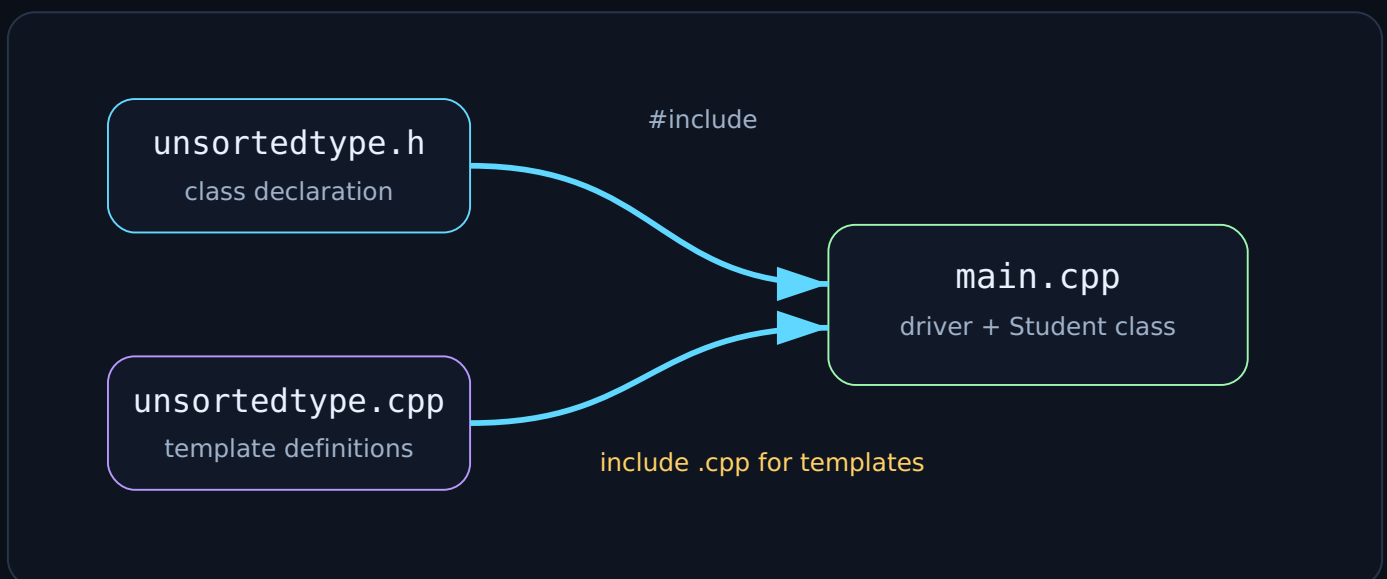
| Task Pattern        | C++ Tool Needed                                             | First Move                                           | Memory Concern                                        | Common Trap                                                               |
|---------------------|-------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------|
| Header/source split | <code>.h</code> , <code>.cpp</code> , <code>main.cpp</code> | Put class declaration in header and tests in driver. | All files must be available to compiler.              | Template definitions hidden in <code>.cpp</code> can cause linker errors. |
| Template list       | <code>template &lt;class T&gt;</code>                       | Write operations using type <code>T</code> .         | <code>new T[SIZE]</code> needs a default constructor. | Forgetting required operators for custom class.                           |
| Array insertion     | Index + size counter                                        | Check <code>IsFull()</code> .                        | Stay inside capacity.                                 | Writing to <code>data[currentSize]</code> when full.                      |
|                     | <code>while</code> loop + flag                              | Start at index 0.                                    | No new memory.                                        |                                                                           |

|                            |                                   |                                  |                                            |                                                     |
|----------------------------|-----------------------------------|----------------------------------|--------------------------------------------|-----------------------------------------------------|
| Linear search              |                                   |                                  |                                            | Not initializing found to false.                    |
| Delete from unsorted array | Search + last-element replacement | Find target index.               | Only logical size changes.                 | Expecting the original order to stay unchanged.     |
| Traversal cursor           | Reset() + GetNext()               | Reset before loop.               | Do not call more than Length() times.      | Printing garbage by over-traversing.                |
| Simple class object        | class , constructors, methods     | Identify fields: ID, name, CGPA. | Object array requires default constructor. | Overloading == using all fields instead of only ID. |

### 3.4 Lab Decision Tree

1. **Am I using the given ADT?** Create an object: `UnsortedType<int> intList;`
2. **Do I need to insert?** Call `Insert(value)` . Do not manually touch `data` .
3. **Do I need to print?** Call `Reset()` , loop from `0` to `Length()-1` , and call `GetNext(item)` .
4. **Do I need to search?** Declare `bool found;` , call `Search(value, found)` , then print based on the flag.
5. **Do I need to delete?** Call `Delete(value)` . Remember: the last item may move into the deleted slot.
6. **Am I storing students?** Build a `Student` class with default constructor, parameterized constructor, `operator==` , and `Print()` .

### 3.5 Required File Structure



| File | Contains | Lab rule |
|------|----------|----------|
|------|----------|----------|

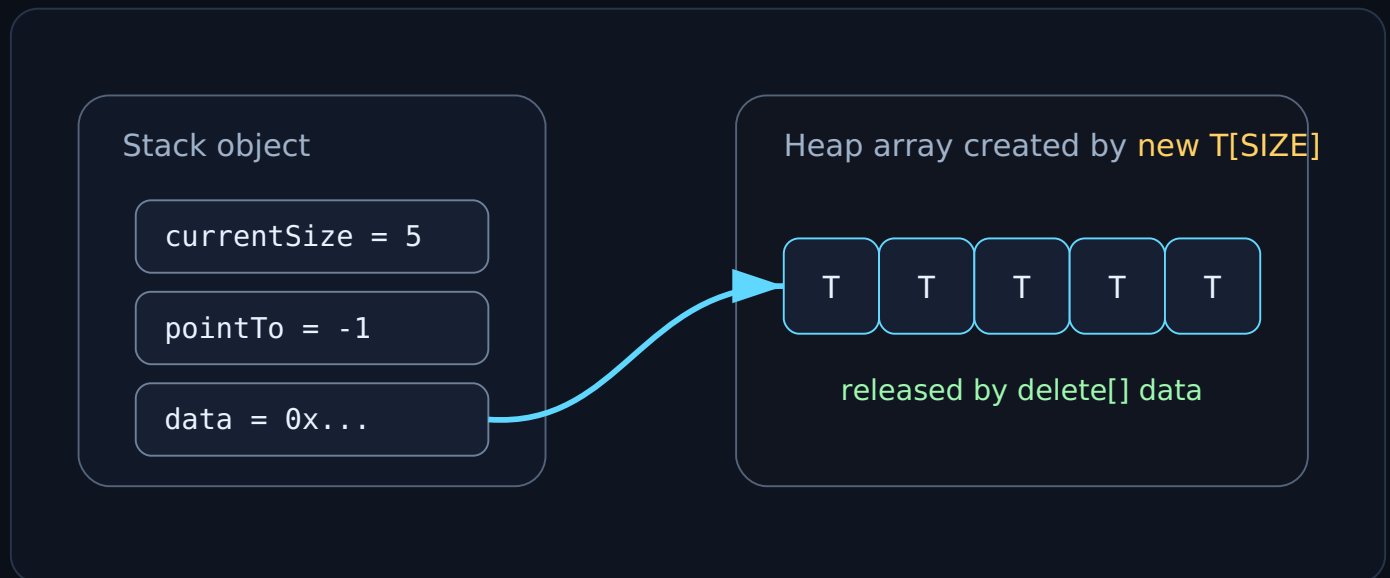
|                  |                                                                                      |                              |
|------------------|--------------------------------------------------------------------------------------|------------------------------|
| unsortedtype.h   | Include guards, SIZE , template class declaration, prototypes.                       | Given by lab. Do not edit.   |
| unsortedtype.cpp | Function definitions for constructor, destructor, insert, search, delete, traversal. | Given by lab. Do not edit.   |
| main.cpp         | Driver code, helper print functions, Student class, task sequence.                   | This is the file you create. |

### Important template note

In standard C++, template function definitions usually need to be visible where the template is instantiated. Because the lab provides template definitions in `unsortedtype.cpp` and says not to edit the given files, the driver includes `#include "unsortedtype.cpp"` . This avoids linker errors while preserving the instructor's files.

## 4. Concept and Implementation Foundation

### 4.1 Internal Memory Model



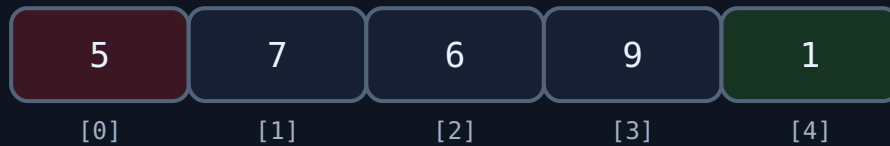
The object itself lives wherever you create it, usually on the stack inside `main()`. Its `data` member is only a pointer. The actual list storage is a heap array of 5 elements. The destructor releases that heap array when the object dies.

| Member                   | Role                                                 | Example state                            |
|--------------------------|------------------------------------------------------|------------------------------------------|
| <code>data</code>        | Pointer to heap array of <code>SIZE</code> elements. | Points to slots <code>[0]...[4]</code> . |
| <code>currentSize</code> | Number of valid items currently stored.              | After inserting 5 values, it is 5.       |
| <code>pointTo</code>     | Traversal cursor used by <code>GetNext()</code> .    | <code>Reset()</code> sets it to -1.      |

### 4.2 Why Delete Reorders the List

The deletion function does not shift every later item left. Instead, it copies the last valid item into the deleted item's slot, then reduces `currentSize`. This is fast after search, but it does not preserve order.

Before Delete(5): found at index 0, last item is 1



After Delete(5): 1 moves to index 0, currentSize becomes 4



### 4.3 Student Object Requirement

The list stores `Student` objects exactly the way it stores integers, but the object type must cooperate with the generic list. Because `UnsortedType` uses `new T[SIZE]`, `Student` must have a default constructor. Because `Search` and `Delete` use `==`, `Student` must overload `operator==`. Because the task asks to print records, `Student` needs a print function.



## 5. Algorithm Workflows

Algorithm: Insert into Unsorted Array List

**Input:** value

**Output:** Value inserted or full-list error printed

1. If `currentSize == SIZE`, print Error: List is full.
2. Otherwise store `value` at `data[currentSize]`.
3. Increase `currentSize` by 1.

Algorithm: Search in Unsorted Array List

**Input:** value

**Output:** `found` becomes true or false

1. Set `found = false`.
2. Start scanning from index 0.
3. If `data[i] == value`, set `found = true` and stop.
4. If the loop ends without a match, `found` stays false.

Algorithm: Delete from Unsorted Array List

**Input:** value

**Output:** Item deleted or not-found error printed

1. Scan to find the index of `value`.
2. If not found, print the lab error message.
3. If found, copy `data[currentSize - 1]` into the found position.
4. Reduce `currentSize` by 1.

Algorithm: Print List Using Reset and GetNext

**Input:** A list object

**Output:** All logical items printed

1. Call `Reset()` to set the cursor to -1.
2. Loop exactly `Length()` times.
3. Inside the loop call `GetNext(item)`.
4. Print the received item.

#### Algorithm: Delete Student by ID

**Input:** A list of `Student` objects and an ID

**Output:** Record with matching ID removed

1. Create a temporary student with the target ID.
2. Make `operator==` compare IDs only.
3. Call `studentList.Delete(target)` .
4. Print the resulting list using each student's `Print()` function.

## 6. Complete Lab Task Solutions

### 6.1 Task Statement

Create `main.cpp`. Use the given `UnsortedType` header and source files without changing them. Perform all integer-list operations in the exact order from the lab sheet. Then create a `Student` class, insert five records, delete the record with ID `15467`, and print the remaining records.

### 6.2 Implementation Plan

| Stage | What to write                                  | Reason                                                                       |
|-------|------------------------------------------------|------------------------------------------------------------------------------|
| 1     | Create helper <code>PrintIntegerList</code> .  | Printing requires <code>Reset()</code> and repeated <code>GetNext()</code> . |
| 2     | Create helper <code>PrintSearchResult</code> . | Keeps the repeated search output clean.                                      |
| 3     | Create helper <code>PrintFullStatus</code> .   | Matches required wording: full or not full.                                  |
| 4     | Create <code>Student</code> .                  | Stores ID, name, CGPA and supports deletion by ID.                           |
| 5     | Write <code>main()</code> in lab-table order.  | Output sequence must match the expected output.                              |

## 6.3 Given File: unsortedtype.h

unsortedtype.h

```
#ifndef UNSORTEDTYPE_H
#define UNSORTEDTYPE_H

const int SIZE = 5;

template <class T>
class UnsortedType
{
private:
    T *data;
    int currentSize;
    int pointTo;

public:
    UnsortedType();
    ~UnsortedType();
    int Length();
    bool IsFull();
    void MakeEmpty();
    void Insert(T value);
    void Search(T value, bool &found);
    void Delete(T value);
    void GetNext(T &value);
    void Reset();
};

#endif // UNSORTEDTYPE_H
```

## 6.4 Given File: unsortedtype.cpp

unsortedtype.cpp - constructor, destructor, Length

```
#include "unsortedtype.h"
#include <iostream>
using namespace std;

template <class T>
UnsortedType<T>::UnsortedType()
{
    data = new T[SIZE];
    currentSize = 0;
    pointTo = -1;
}

template <class T>
UnsortedType<T>::~~UnsortedType()
{
    delete[] data;
}

template <class T>
int UnsortedType<T>::Length()
{
    return currentSize;
}
```

## unsortedtype.cpp - IsFull, MakeEmpty, Insert, Search

```
template <class T>
bool UnsortedType<T>::IsFull()
{
    return (SIZE == currentSize);
}

template <class T>
void UnsortedType<T>::MakeEmpty()
{
    currentSize = 0;
}

template <class T>
void UnsortedType<T>::Insert(T value)
{
    if (IsFull())
    {
        cout << "Error: List is full" << endl;
    }
    else
    {
        data[currentSize] = value;
        currentSize++;
    }
}

template <class T>
void UnsortedType<T>::Search(T value, bool &found)
{
    found = false;
    int i = 0;

    while (i < currentSize)
    {
        if (data[i] == value)
        {
            found = true;
            break;
        }
        else
        {
            i++;
        }
    }
}
```

#### unsortedtype.cpp - Delete, GetNext, Reset

```
template <class T>
void UnsortedType<T>::Delete(T value)
{
    bool found = false;
    int i = 0;

    while (i < currentSize)
    {
        if (data[i] == value)
        {
            found = true;
            break;
        }
        else
        {
            i++;
        }
    }

    if (found)
    {
        data[i] = data[currentSize - 1];
        currentSize--;
    }
    else
    {
        cout << "Error: Item could not be found in the list" << endl;
    }
}

template <class T>
void UnsortedType<T>::GetNext(T &value)
{
    pointTo++;
    value = data[pointTo];
}

template <class T>
void UnsortedType<T>::Reset()
{
    pointTo = -1;
}
```

## 6.5 Driver File You Create: main.cpp

#### main.cpp - includes

```
#include <iostream>
#include <string>
#include "unsortedtype.h"
#include "unsortedtype.cpp" // Needed because UnsortedType is a template class.
using namespace std;
```

## main.cpp - Student class

```
class Student
{
private:
    int id;
    string name;
    double cgpa;

public:
    Student()
    {
        id = 0;
        name = "";
        cgpa = 0.0;
    }

    Student(int studentId, string studentName, double studentCgpa)
    {
        id = studentId;
        name = studentName;
        cgpa = studentCgpa;
    }

    bool operator==(const Student &other) const
    {
        return id == other.id;
    }

    void Print() const
    {
        cout << id << ", " << name << ", " << cgpa << endl;
    }
};
```

## main.cpp - helper functions

```
void PrintIntegerList(UnsortedType<int> &list)
{
    int item;
    list.Reset();

    for (int i = 0; i < list.Length(); i++)
    {
        list.GetNext(item);
        cout << item << " ";
    }
    cout << endl;
}

void PrintStudentList(UnsortedType<Student> &list)
{
    Student student;
    list.Reset();

    for (int i = 0; i < list.Length(); i++)
    {
        list.GetNext(student);
        student.Print();
    }
}

void PrintSearchResult(UnsortedType<int> &list, int item)
{
    bool found;
    list.Search(item, found);

    if (found)
    {
        cout << "Item is found" << endl;
    }
    else
    {
        cout << "Item is not found" << endl;
    }
}

void PrintFullStatus(UnsortedType<int> &list)
{
    if (list.IsFull())
    {
        cout << "List is full" << endl;
    }
    else
    {
        cout << "List is not full" << endl;
    }
}
```



## main.cpp - task sequence

```
int main()
{
    UnsortedType<int> intList;

    intList.Insert(5);
    intList.Insert(7);
    intList.Insert(6);
    intList.Insert(9);

    PrintIntegerList(intList);
    cout << intList.Length() << endl;

    intList.Insert(1);
    intList.Insert(12);

    PrintIntegerList(intList);

    PrintSearchResult(intList, 4);
    PrintSearchResult(intList, 5);
    PrintSearchResult(intList, 9);
    PrintSearchResult(intList, 10);

    PrintFullStatus(intList);

    intList.Delete(5);
    PrintFullStatus(intList);
    PrintIntegerList(intList);

    intList.Delete(1);
    PrintIntegerList(intList);

    intList.Delete(6);
    PrintIntegerList(intList);

    intList.Delete(16);

    cout << endl;

    UnsortedType<Student> studentList;

    studentList.Insert(Student(15234, "Jon", 2.6));
    studentList.Insert(Student(13732, "Tyrion", 3.9));
    studentList.Insert(Student(13569, "Sandor", 1.2));
    studentList.Insert(Student(15467, "Ramsey", 3.8));
    studentList.Insert(Student(16285, "Arya", 3.1));

    studentList.Delete(Student(15467, "", 0.0));
    PrintStudentList(studentList);

    return 0;
}
```

## 6.6 Expected Terminal Output

Windows 11 PowerShell Sample Run

```
PS C:\Users\User\CSE225L\lab05> g++ main.cpp -o lab05
PS C:\Users\User\CSE225L\lab05> .\lab05.exe
5 7 6 9
4
Error: List is full
5 7 6 9 1
Item is not found
Item is found
Item is found
Item is not found
List is full
List is not full
1 7 6 9
9 7 6
9 7
Error: Item could not be found in the list

15234, Jon, 2.6
13732, Tyrion, 3.9
13569, Sandor, 1.2
16285, Arya, 3.1
```

## 6.7 Code Explanation

| Code part                                | Purpose                                                         | What to watch                                                       |
|------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------|
| <code>#include "unsortedtype.cpp"</code> | Makes template definitions visible to <code>main.cpp</code> .   | Use this because the lab says not to edit the given template files. |
| <code>Student()</code>                   | Default constructor needed for new <code>Student[SIZE]</code> . | Without it, the template array allocation may fail.                 |
| <code>operator==</code>                  | Allows Search and Delete to compare students.                   | Compare only IDs so temporary delete target works.                  |
| <code>PrintIntegerList</code>            | Prints list through public ADT methods.                         | Never access <code>data</code> directly because it is private.      |
| <code>Delete(5)</code>                   | Removes 5 and moves the last item 1 to the front.               | This is why output becomes <code>1 7 6 9</code> .                   |

## 6.8 Dry Run: Integer List

| Operation         | Logical list after operation | Output  |
|-------------------|------------------------------|---------|
| Insert 5, 7, 6, 9 | 5 7 6 9                      | none    |
| Print             | 5 7 6 9                      | 5 7 6 9 |
| Length            | 5 7 6 9                      | 4       |

|                    |                        |                                            |
|--------------------|------------------------|--------------------------------------------|
| Insert 1           | 5 7 6 9 1              | none                                       |
| Insert 12          | unchanged because full | Error: List is full                        |
| Search 4, 5, 9, 10 | unchanged              | not found, found, found, not found         |
| Delete 5           | 1 7 6 9                | none                                       |
| Delete 1           | 9 7 6                  | none                                       |
| Delete 6           | 9 7                    | none                                       |
| Delete 16          | 9 7                    | Error: Item could not be found in the list |

## 6.9 Dry Run: Student List

| Step                | List state                        | Why                                                          |
|---------------------|-----------------------------------|--------------------------------------------------------------|
| Insert five records | Jon, Tyrion, Sandor, Ramsey, Arya | Each record is stored in the next available slot.            |
| Delete ID 15467     | Jon, Tyrion, Sandor, Arya         | Ramsey is at index 3, Arya is last and moves into that slot. |
| Print               | Four records printed              | PrintStudentList loops through Length() .                    |

## 6.10 Edge Cases

| Case                                      | What happens                                        | Safe handling                  |
|-------------------------------------------|-----------------------------------------------------|--------------------------------|
| Insert into full list                     | Prints Error: List is full .                        | Do not increase currentSize .  |
| Delete missing item                       | Prints not-found error.                             | List stays unchanged.          |
| Delete last item                          | Copies last item onto itself, then decrements size. | Still valid.                   |
| Print after forgetting Reset ( )          | Cursor may continue from old position.              | Always reset before traversal. |
| Custom object without default constructor | new T[SIZE] may not compile.                        | Provide Student() .            |

## 6.11 Complexity Analysis

| Operation | Time | Why                     |
|-----------|------|-------------------------|
| Length()  | O(1) | Returns stored counter. |
| IsFull()  | O(1) | Compares two integers.  |

|           |      |                                        |
|-----------|------|----------------------------------------|
| Insert()  | O(1) | Writes to next slot.                   |
| Search()  | O(n) | May scan every valid item.             |
| Delete()  | O(n) | Search dominates; replacement is O(1). |
| GetNext() | O(1) | Moves cursor by one.                   |

## 6.12 Lab-Viva Notes

| Question                                         | Short answer                                                             |
|--------------------------------------------------|--------------------------------------------------------------------------|
| Why is this list called unsorted?                | Because the code does not maintain ascending or descending order.        |
| Why does deleting 5 change the first value to 1? | The delete function replaces the deleted item with the last valid item.  |
| Why does Student need operator== ?               | The generic search and delete functions compare objects using == .       |
| Why do we need Reset() before printing?          | It sets the traversal cursor back to the position before the first item. |
| What is the capacity of this list?               | 5, because <code>const int SIZE = 5</code> .                             |

## 7. Debugging and Common Errors

### Linker error: undefined reference to `UnsortedType<int>::Insert`

Cause: template definitions were placed in `unsortedtype.cpp` but not visible to `main.cpp`.  
Fix for this lab: include `unsortedtype.cpp` in `main.cpp`, or move template definitions into the header in a redesigned version.

### Garbage output while printing

Cause: calling `GetNext()` more times than `Length()`, or forgetting `Reset()`. Fix: always reset and loop exactly `list.Length()` times.

### Student list does not compile

Cause: `Student` lacks a default constructor, or `operator==` is missing. Fix: write `Student()` and `bool operator==(const Student &other) const`.

### Expected order seems wrong after delete

Cause: this implementation uses swap-with-last deletion. It is intentional for an unsorted list.  
Fix: do not shift elements unless the instructor asks for order-preserving deletion.

### Full-list insertion changes list accidentally

Cause: code writes into the array even when full. The given `Insert` function is safe because it prints an error and skips insertion.

## 8. Testing Checklist

|   | Test                                   | Expected result                               |
|---|----------------------------------------|-----------------------------------------------|
| ✓ | Insert exactly four integers and print | 5 7 6 9                                       |
| ✓ | Print length after four insertions     | 4                                             |
| ✓ | Fill capacity and insert one extra     | Error: List is full                           |
| ✓ | Search missing values 4 and 10         | Item is not found                             |
| ✓ | Search present values 5 and 9          | Item is found                                 |
| ✓ | Delete 5, 1, 6 in sequence             | List states become 1 7 6 9, 9 7 6, 9 7.       |
| ✓ | Delete 16                              | Not-found error.                              |
| ✓ | Insert and delete Student ID 15467     | Remaining records: Jon, Tyrion, Sandor, Arya. |

## 9. Complexity Summary

| Method          | Best             | Average          | Worst            | Space                  |
|-----------------|------------------|------------------|------------------|------------------------|
| Insert          | $O(1)$           | $O(1)$           | $O(1)$           | $O(1)$ extra           |
| Search          | $O(1)$           | $O(n)$           | $O(n)$           | $O(1)$                 |
| Delete          | $O(1)$           | $O(n)$           | $O(n)$           | $O(1)$                 |
| Traversal print | $O(n)$           | $O(n)$           | $O(n)$           | $O(1)$                 |
| Constructor     | $O(\text{SIZE})$ | $O(\text{SIZE})$ | $O(\text{SIZE})$ | $O(\text{SIZE})$       |
| Destructor      | $O(1)$           | $O(1)$           | $O(1)$           | Frees $O(\text{SIZE})$ |

Here `n` means `currentSize`, not the fixed capacity constant. In this lab capacity is 5, but we still analyze it generally because the same ADT idea can scale.

## 10. Quick Revision Sheet

### Core Syntax Patterns

- `UnsortedType<int> list;`
- `list.Insert(5);`
- `bool found; list.Search(5, found);`
- `list.Delete(5);`
- `list.Reset();`  
`list.GetNext(item);`

### Pointer Safety Rules

- `new T[SIZE]` needs `delete[]` data.
- Never access private data from `main()`.
- Do not call `GetNext()` beyond `Length()`.
- `MakeEmpty()` changes logical size only.

| Need to...           | Use...                               | Remember...                                      |
|----------------------|--------------------------------------|--------------------------------------------------|
| Add an item          | <code>Insert(value)</code>           | Fails if list is full.                           |
| Find an item         | <code>Search(value, found)</code>    | Pass <code>found</code> by reference.            |
| Remove an item       | <code>Delete(value)</code>           | Order may change.                                |
| Print all items      | <code>Reset() + GetNext()</code>     | Loop exactly <code>Length()</code> times.        |
| Delete student by ID | <code>operator==</code> comparing ID | Temporary target can have blank name and 0 CGPA. |

### Final Submission Checklist

|   |                                                                                               |                                                        |
|---|-----------------------------------------------------------------------------------------------|--------------------------------------------------------|
| ✓ | <code>main.cpp</code> exists and compiles.                                                    | No missing include or linker issue.                    |
| ✓ | Given <code>.h</code> and <code>.cpp</code> files are not edited.                             | Lab requirement preserved.                             |
| ✓ | Output wording matches expected output.                                                       | Especially full/not full and found/not found messages. |
| ✓ | <code>Student</code> has ID, name, CGPA, print function, constructors, and equality overload. | Object list works.                                     |
| ✓ | Code is tested after each deletion.                                                           | Deletion order is understood.                          |

## 11. Lab Coverage Audit

| Page / Slide Range | Task or Topic Found                                                                      | Where Covered in Guide       | Status                       | Notes                                           |
|--------------------|------------------------------------------------------------------------------------------|------------------------------|------------------------------|-------------------------------------------------|
| Page 1             | Lab objectives: array-based unsorted List ADT and Student records                        | Sections 1, 3, 4, 6          | Explained                    | All objectives mapped to implementation skills. |
| Page 2             | <code>unsortedtype.h</code> ; constructor, destructor, <code>Length</code>               | Sections 3.5, 4.1, 6.3, 6.4  | Explained, Implemented       | Template visibility issue noted.                |
| Page 3             | <code>IsFull</code> , <code>MakeEmpty</code> , <code>Insert</code> , <code>Search</code> | Sections 5, 6.4, 6.8, 9      | Explained, Traced            | Full-list behavior and linear search covered.   |
| Page 4             | <code>Delete</code> , <code>GetNext</code> , <code>Reset</code>                          | Sections 4.2, 5, 6.4, 6.8, 7 | Explained, Traced, Corrected | Delete reorder behavior emphasized.             |
| Page 5             | Integer tasks and expected output                                                        | Sections 6.1-6.12, 8, 10     | Implemented, Tested          | Terminal output matches operation sequence.     |
| Page 5             | Student class and record deletion task                                                   | Sections 4.3, 6.5, 6.9, 6.12 | Implemented, Tested          | Equality overload by ID makes deletion work.    |

End of guide. Keep this one in your lab folder, Boss. The tiny array goblin has been domesticated. 